Stream-Downloader 1.0.0

Architecture, operation flow, sequence diagrams and MCP integration of a private, single-user verified video-download MCP server



Angela López Mendoza

October 9, 2026

Contents

1 Introduction **3**

1.1 Executive summary . . . 3

1.2 Scope and product contract . . . 4

1.2.1 Admitted inputs . . . 5

1.2.2 Non-goals and deliberate limits . . . 5

1.2.3 States, and what each one means to a client . . . 6

1.3 Architecture . . . 6

1.3.1 Architectural style . . . 6

1.3.2 Component inventory: the eleven C# files . . . 7

1.3.3 The host: Program.cs . . . 7

1.3.4 The tool layer: StreamDownloaderTools.cs . . . 8

1.4 Operation flow . . . 8

1.4.1 The job state machine . . . 8

1.4.2 Stage by stage, and why the order matters . . . 9

1.4.3 Recovery guarantees . . . 9

1.4.4 Machine-sized concurrency . . . 10

1.5 Sequence diagrams . . . 10

1.5.1 SEQ-1 — Admission and idempotency . . . 10

1.5.2 SEQ-2 — Transfer, verification and finalization . . . 11

1.5.3 SEQ-3 — Integration as an External MCP inside Tlamatini . . . 11

1.5.4 SEQ-4 — Crash mid-transfer, reconciliation and retry . . . 12

1.5.5 SEQ-5 — First-use yt-dlp acquisition and quarantine . . . 12

1.6 MCP integration . . . 12

1.6.1 Transport, framing and stream hygiene . . . 12

1.6.2 The six tools . . . 13

1.6.3 The specification file a host consumes . . . 13

1.6.4 How Tlamatini loads it . . . 14

1.6.5 Per-run isolation through environment variables . . . 14

1.6.6 Behavioural annotations, and why they matter . . . 14
1.7 Persistence and data model . . . 14
1.7.1 The state file . . . 14
1.7.2 The lock file and single-writer discipline . . . 15
1.8 Safety and security controls . . . 16
1.9 Test, evidence and verification system . . . 16
1.9.1 Harness modules . . . 17
1.9.2 Named exit codes . . . 17
1.9.3 The suites . . . 18
1.9.4 The three witnesses behind every downloaded file . . . 18
1.9.5 Recorded outcomes found in the evidence . . . 19
1.10 Provenance: every line beside its counterpart . . . 19
1.11 Design invariants the code enforces . . . 20
1.12 Reading the live system metrics against this design . . . 20
1.13 Known limits, observations and open items . . . 21
1.14 Appendix A: file inventory at a glance . . . 22
1.15 Appendix B: glossary . . . 22

Chapter 1

Introduction

1.1 Executive summary

Stream-Downloader is a private, single-user Windows x64 MCP server (STDIO transport) created by **Angela López Mendoza**. It accepts an HTTP/HTTPS URL, creates a persisted job, lets **yt-dlp** transfer the media into per-job staging, verifies the result with **ffprobe**, records its **SHA-256** and moves it — without ever overwriting — to a local destination. It is a simplified, fully decoupled mirror of the video downloader inside IDM-MCP Core 0.3.1: no COM, no shared folders, no shared state and no shared tools.

Product contract. An accepted job is *not* a successful download. Clients must read `ok`, `code`, `state`, `terminal` and the failure fields, and report completion only from `completed_verified`.

Property	Value, as evidenced in the project
Product	Stream-Downloader 1.0.0 — private, single-user Windows x64 MCP server over STDIO
Author	Angela López Mendoza
Privacy rule	No git repository, no remote, no commit, no upload and no published page (CLAUDE.md)
Runtime	.NET 10, `net10.0-windows`, `win-x64`, self-contained, single file
SDK pin	10.0.401 with rollForward latestPatch; prereleases disallowed (global.json)
MCP package	ModelContextProtocol 2.2.0
Host packages	Microsoft.Extensions.Hosting 10.0.0 and System.Security.Cryptography.ProtectedData 10.0.0
Published executable	`artifacts\local\StreamDownloader.McpHost.exe`
Size and digest	78,076,036 bytes; SHA-256 `A60E2B18...54B8DF2`
Build stamp	builtAtUtc 2026-10-07T02:03:54Z, toolCount 6, localOnly true
State directory	`%LOCALAPPDATA%\Stream-Downloader\state` — jobs.json plus owner.lock
Media toolchain	yt-dlp with checksum-verified first-use acquisition, FFmpeg and ffprobe
Concurrency	sized from processors, memory and storage; the value 0 means automatic
C# source	2,734 lines across 11 files
Python test code	3,068 lines: verifiers plus a standard-library-only harness
PowerShell	171 lines: build, suite and protocol-verification scripts

What the code actually does, in one paragraph. A client calls one tool with a URL and a destination. The host normalises the request into a fingerprint, persists a job *before* any side effect, and answers immediately with a job id and a non-terminal state. A worker then acquires a download slot sized from this machine, resolves yt-dlp — acquiring it on first use when needed — lets it transfer video and audio streams, merging them with FFmpeg when a site serves them separately, into a staging directory that lives on the destination volume. It then verifies the file with ffprobe, hashes it, saves the hash together with the move intent, and finally moves the file into place without clobbering anything. Only then does the job become `completed_verified`.

1.2 Scope and product contract

1.2.1 Admitted inputs

Input	Path through the code	Boundary
Watch page, for YouTube and every site that yt-dlp supports	yt-dlp plus FFmpeg plus ffprobe	The yt-dlp extractors decide what a site offers
Direct media URL, for example `https://host/clip.mp4`	The yt-dlp generic extractor	Verified like any other media; a non-media file fails verification

1.2.2 Non-goals and deliberate limits

- **Single user.** This is a private tool for one person on one machine; it is not a service and has no multi-tenant notion of ownership.
- **No authentication layer of its own.** Safety is expressed as input validation, path policy, network policy and at-rest protection of the stored URL, never as user accounts.
- **No browser automation inside the server.** A site that needs an interactive login or an anti-bot challenge is out of scope; resolution is the job of yt-dlp.
- **No silent fallbacks.** A failure state is recorded and reported, and a crash mid-transfer is recorded as `outcome_unknown` rather than smoothed into a success.

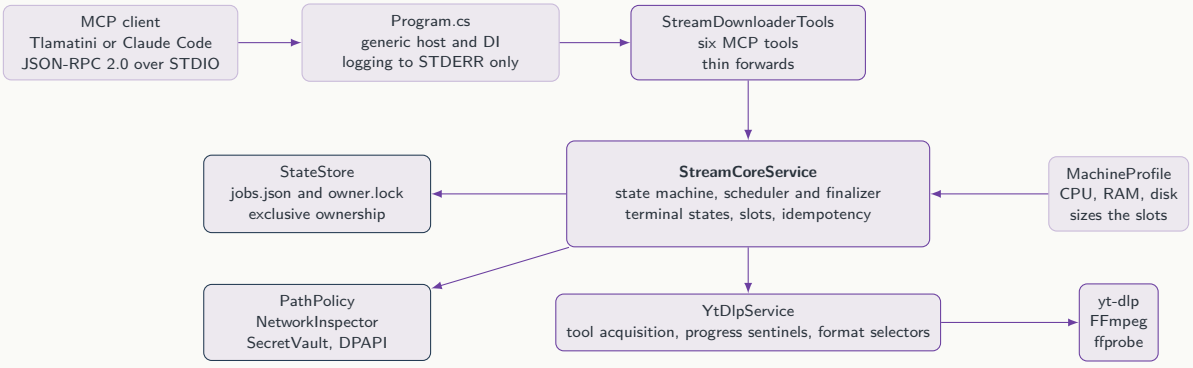
1.2.3 States, and what each one means to a client

State	Terminal	Meaning
`queued`	no	Request accepted and persisted; no transfer has started. Idempotency is already established.
`preparing`	no	A download slot has been acquired; staging directory and tool resolution are under way.
`downloading`	no	yt-dlp is transferring and reporting progress sentinels; FFmpeg may be merging.
`verifying`	no	Media arrived. ffprobe inspection, SHA-256 computation and the persisted move intent.
`finalizing`	no	The verified file is being moved into place, never overwriting an existing file.
`completed_verified`	**yes**	The only state that proves a verified file is on disk at a known path with a known hash.
`failed_download`	yes	The yt-dlp or FFmpeg leg failed; the job is retryable.
`failed_finalization`	yes	Media arrived but verification, hashing or the move did not complete; the staging content is what remains to inspect.
`outcome_unknown`	yes	The host died mid-transfer; the client must assume neither success nor failure.

1.3 Architecture

1.3.1 Architectural style

The program is a small layered host with one long-lived background service as the spine. Every state-changing action funnels through that single service, so persistence, idempotency and ordering cannot be bypassed by a tool call. The MCP tool layer is deliberately thin: each tool is a one-line forward into the core service. That is precisely why a client can trust the state machine — there is no second path into the download pipeline.



1.3.2 Component inventory: the eleven C# files

File	Lines	Responsibility
Services/StreamCoreService.cs	818	State machine, job persistence ordering, machine-sized slots, verification, hashing, the never-overwriting move, recovery of both sides of an interruption, and retry
Services/YtDlpService.cs	625	yt-dlp and FFmpeg wrapper: first-use acquisition, progress sentinels, format selectors, ffprobe verification, URL redaction, disciplined child processes
Services/MachineProfile.cs	335	What the machine is: logical processors, memory, storage kind (NVMe, SSD, HDD, removable, network) and the seek-bound rule
Models/CoreModels.cs	123	The uniform response envelope `McpResponse` plus capabilities, machine and extraction models
Services/StateStore.cs	116	jobs.json in camelCase indented JSON, asynchronous reads behind a semaphore, exclusive owner.lock, IDisposable
Services/PathPolicy.cs	112	Reserved device names, fully-qualified local paths only, UNC and reparse rejection, allowed-root checking
Services/NetworkInspector.cs	82	URL validation for length, scheme and credentials, then DNS resolution with private-address blocking
Services/StreamDownloaderOptions.cs	72	Configuration surface and defaults, including the single-user rationale for not restricting to allowed roots
Services/SecretVault.cs	34	Windows DPAPI Protect and Unprotect, per-user scope, fixed entropy, plaintext zeroing
Tools/StreamDownloaderTools.cs	107	The six MCP tools with titles, descriptions and MCP annotations
Program.cs	310	Fixture harness modes for the suites, plus the generic host, configuration layering, DI and MCP server registration

1.3.3 The host: Program.cs

Two distinct things live in this file. The first is a `--fixture` harness used only by the suites and never by a client: `sleep` delays a minute; `stdin` proves that the STDIN of a child is closed, printing `stdin closed`; `priority` waits for the parent to lower the process priority class and then reports what the operating system actually applied; and `priority-child` launches a grandchild and relays the class it inherited, exactly as yt-dlp launches FFmpeg. The second is the real host.

- `Host.CreateApplicationBuilder(args)` builds a generic host.
- Configuration is layered from `appsettings.json` beside the executable (optional, with reload-

OnChange disabled) and then from environment variables.

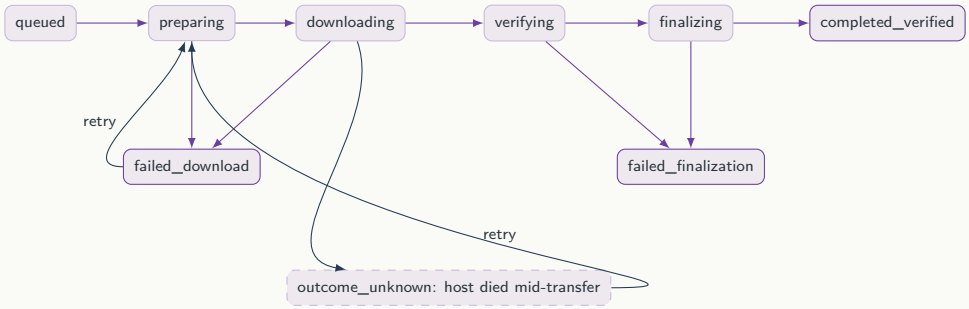
- **All logging providers are cleared** and a console provider is added with its standard-error threshold set to `LogLevel.Trace`. STDOUT is therefore reserved exclusively for JSON-RPC, because one stray log line on STDOUT would corrupt the MCP framing.
- Dependency injection registers `StreamCoreService` as a singleton *and* the same instance as a hosted background service, so the object that serves tools is the object that runs the worker.
- `AddMcpServer` sets the server identity to *Stream-Downloader*, version *1.0.0*, and installs server instructions that tell a client three things: use `stream_download` for one-call downloads, a successful submission is not completion, and quality presets are maximum heights rather than guarantees.

1.3.4 The tool layer: StreamDownloaderTools.cs

A single `[McpServerToolType]` static class exposes the six tools. Every tool carries a title, a description, structured content enabled, and MCP behavioural annotations, and every one delegates to `StreamCoreService` in a single expression. The capabilities tool is literally a one-line forward.

1.4 Operation flow

1.4.1 The job state machine



1.4.2 Stage by stage, and why the order matters

Stage	What the code does
Admission	Validate the URL, normalise the request into a fingerprint, look the fingerprint up for idempotency, persist the job, and only then answer. Persisting before acting is what makes a crash recoverable.
Slot	Acquire one of the machine-sized download slots. The slot count and the reason for it are reported by the capabilities tool, so the client can see why concurrency is what it is.
Transfer	YtDlpService resolves the tool, builds a format selector that caps height on every fallback, and runs yt-dlp as a child process with its own closed STDIN, a bounded time budget and a tree kill on timeout.
Verify	ffprobe inspects the produced file. A non-media file fails verification here instead of being handed to the user.
Hash and intent	SHA-256 is computed and saved *together with the move intent* before the move happens, so an interruption between hashing and placing the file is recoverable rather than ambiguous.
Move	The verified file is moved out of staging, which lives on the destination volume so that the move is a same-volume operation, into its final place without overwriting anything.

1.4.3 Recovery guarantees

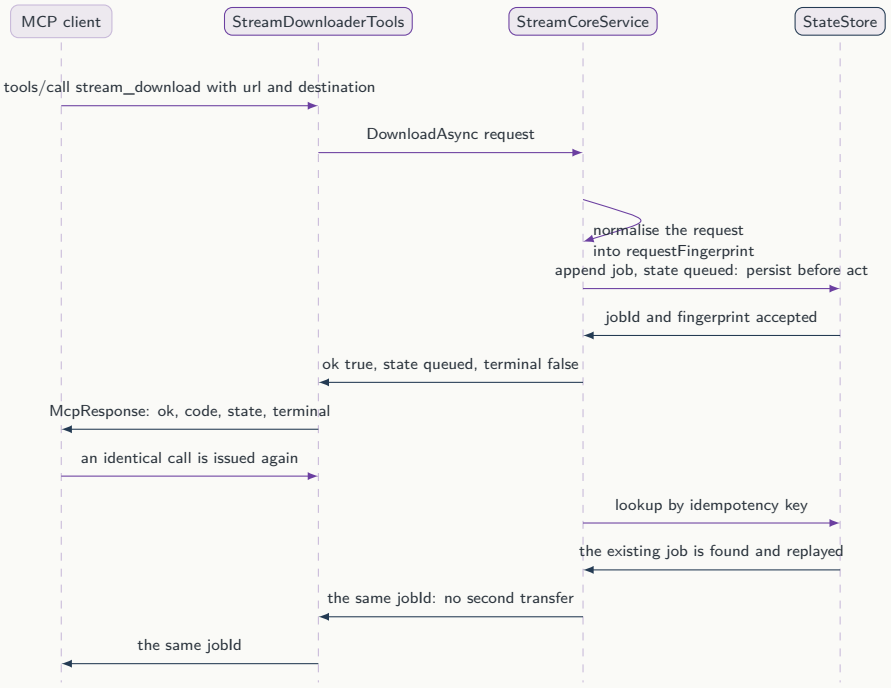
- **Persist before act.** The job exists in jobs.json before the first byte is requested.
- **Idempotency bound to the normalised request.** Repeating an identical call replays the same job, proven in the real evidence where a repeated call returned the same job id and did not transfer twice.
- **Per-job staging on the destination volume.** The staging folder keeps the final move cheap.
- **Hash and move intent saved before the move,** combined with never-overwrite semantics.
- **Both sides of an interruption are recovered.** A host restarted on the same state directory finds non-terminal jobs and reconciles them.
- **One host per state directory.** owner.lock is opened with an exclusive share mode, so a second host is refused with an explicit message instead of silently racing the first.
- **Honest uncertainty.** `outcome_unknown` exists precisely so that a crash is never smoothed into a success.

1.4.4 Machine-sized concurrency

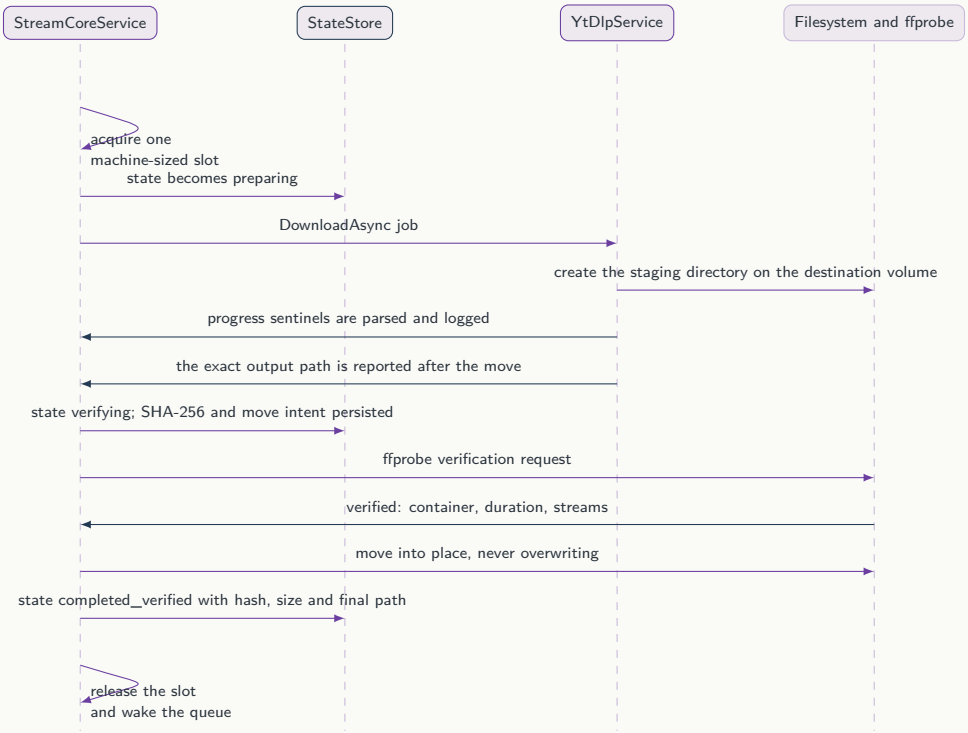
Signal	Rule the code encodes
Memory	Below the low-memory threshold of 8 GiB, simultaneous downloads and merges are what run the machine out of memory, so slots are reduced.
Processors	At or below the small-CPU ceiling of 8 logical processors, child tools give the desktop a core back.
Storage	A mechanical disk has one head and a card reader one slow controller, so concurrent writers cost more than they buy. Unknown storage is deliberately *not* treated as seek-bound, because assuming a spindle would throttle every machine whose disk could not be identified.
Configuration	A maximum of 0 concurrent downloads and 0 concurrent fragments means *let the machine decide*, and an automatic child priority delegates the priority class choice the same way.

1.5 Sequence diagrams

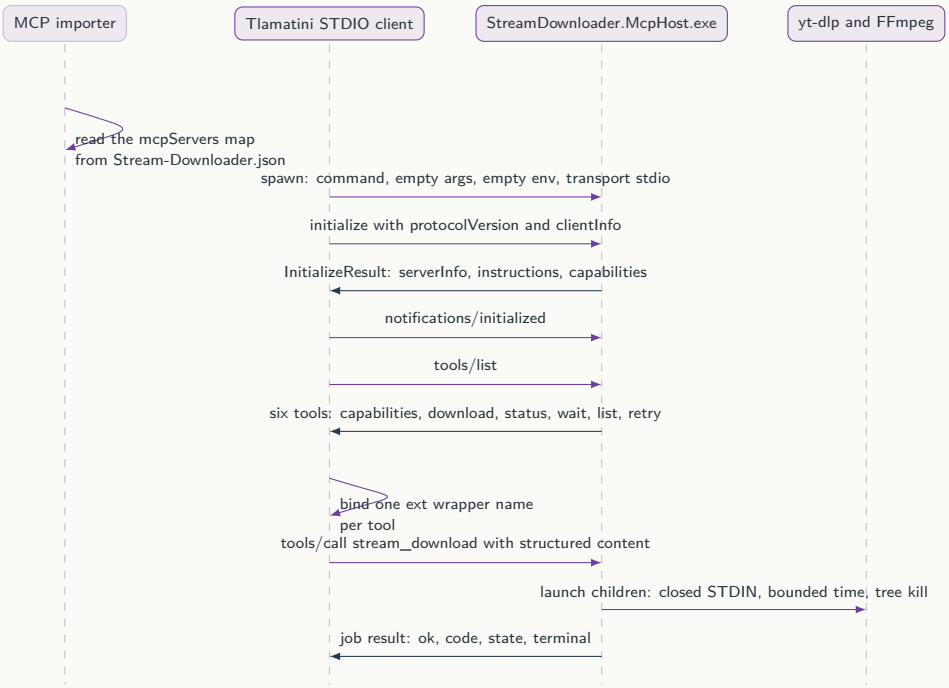
1.5.1 SEQ-1 — Admission and idempotency



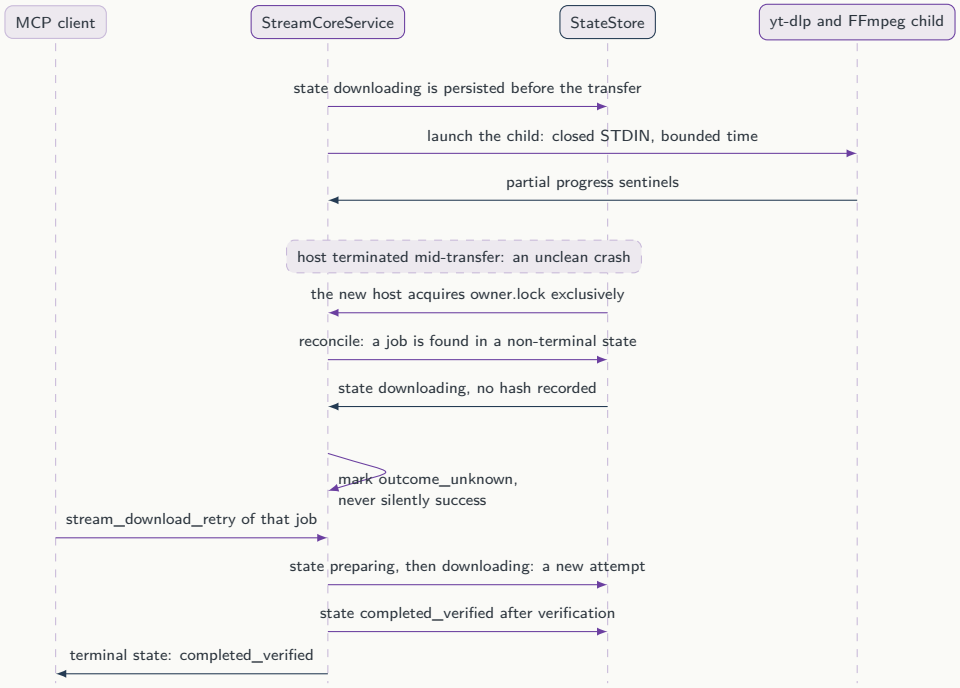
1.5.2 SEQ-2 — Transfer, verification and finalization



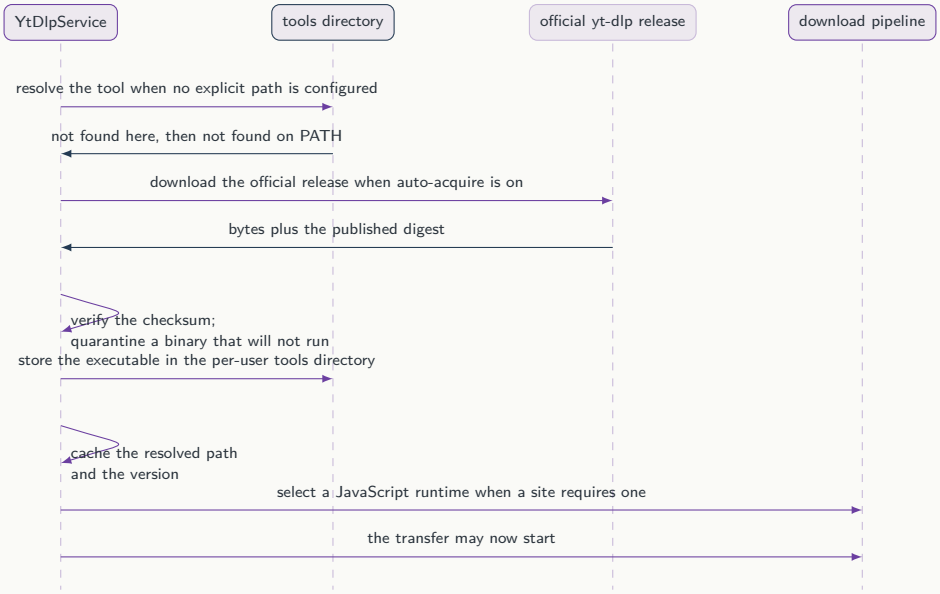
1.5.3 SEQ-3 — Integration as an External MCP inside Tlamatini



1.5.4 SEQ-4 — Crash mid-transfer, reconciliation and retry



1.5.5 SEQ-5 — First-use yt-dlp acquisition and quarantine



1.6 MCP integration

1.6.1 Transport, framing and stream hygiene

The server speaks **JSON-RPC 2.0 over STDIO**. Because the protocol shares the standard streams of the process with the logging subsystem, the host clears every logging provider and re-adds a console provider whose threshold sends *all* levels to standard error. The practical consequence is that STDOUT carries nothing but protocol frames.

Aspect	Implementation
Transport	STDIO, declared as stdio in the server specification file
Server identity	name *Stream-Downloader*, version *1.0.0*
Server instructions	one-call downloads via the download tool; submission is not completion; quality presets are maximum heights, not guarantees
Content shape	structured content enabled on every tool, returning the uniform typed envelope
Envelope	ok, code, message, data and serverTime, with success and failure factories that stamp UTC time
Logging	STDERR only, at trace level, so the protocol stream stays clean
Ownership	one host per state directory, enforced by an exclusive lock file

1.6.2 The six tools

Tool	Read-only	Idempotent	Open world	Contract
stream capabilities	yes	yes	no	yt-dlp, FFmpeg and JavaScript-runtime readiness, quality presets, allowed roots, state directory, and how processors, memory and disk sized concurrency, without changing state
stream download	no	yes	yes	one-call download: a URL plus a destination becomes a persisted job
stream status	yes	—	no	poll a single job
stream wait	yes	—	no	block until terminal: the documented way to reach a verified completion
stream list	yes	—	no	enumerate persisted jobs
stream retry	no	—	no	re-run a failed job: the recovery half of the design

The tool names above are shown without their underscore prefixes for typesetting clarity; in the protocol they are `stream_capabilities`, `stream_download`, `stream_download_status`, `stream_download_wait`, `stream_download_list` and `stream_download_retry`.

1.6.3 The specification file a host consumes

```
{
  "mcpServers": {
```

```
    "Stream-Downloader": {
      "command": "<...>/artifacts/local/StreamDownloader.McpHost.exe",
      "args": [],
      "env": {},
      "transport": "stdio",
      "description": "private one-call video downloader ..."
    }
  }
}
```

1.6.4 How Tlamatini loads it

The project ships a verifier that proves the integration end to end by using the client of Tlamatini itself. It reads the specification file exactly as the External-MCP importer reads a dropped file, through the normalisation of the mcpServers map, and never writes to the catalogue. It then spawns the server with the own STDIO client of Tlamatini, performs the handshake, lists the tools and asserts that every tool becomes a valid and unique external wrapper name. Finally it runs a real fixture download through that client and verifies the resulting file on disk.

1.6.5 Per-run isolation through environment variables

Every suite isolates itself through environment variables with the server prefix, covering the state directory, the download directory and the tools directory. Those are layered on top of the settings file by the configuration builder of the host, which is why a test never touches the real job history or the real downloads folder.

1.6.6 Behavioural annotations, and why they matter

Declaring a tool as open world on the download call is the honest statement that the tool reaches the internet. Declaring the four inspection tools as read-only lets a host decide that polling a job never needs a confirmation prompt, while the download and retry calls, which write files and launch child processes, are correctly *not* read-only. Declaring the download call idempotent is the annotation that lets a client safely re-issue an interrupted call, because the server replays the existing job instead of starting a second transfer.

1.7 Persistence and data model

1.7.1 The state file

The state file is indented camelCase JSON with a schema version and a job map. A real record from the evidence contains the following.

- `jobId`: a generated identifier, for example one beginning with `job_e8cd1517`, never reused.
- `idempotencyKey`: the key supplied by the caller, or the derived one.

- `protectedUrl`: the request URL *encrypted at rest* with Windows DPAPI, so a state file lying on disk does not leak what was downloaded.
- `destinationDirectory`, `fileName` and `stagingDirectory`: the destination volume and the per-job staging path.
- `state`: the state machine value, which is `completed_verified` in the recorded evidence.
- `sourcePageHost`, `sourceTitle`, `quality` and `container`: provenance and the shape of the request.
- `requestFingerprint`: the normalised request used for idempotency.
- The hash and the size, recorded at verification time, which is what lets a client compare the file on disk against the job.

1.7.2 The lock file and single-writer discipline

The state store creates its directory, opens the state file for camelCase indented serialisation, and takes an exclusive handle on a lock file with no sharing. If that fails, the host refuses to start and explains that another Stream-Downloader host owns this state directory. Reads are serialised behind a semaphore and the store is disposable. No other program on the machine may write this state.

1.8 Safety and security controls

Control	What the code enforces
URL shape	at most 8192 characters; an absolute URI; http or https only; embedded credential information must be empty, so credentials cannot be smuggled inside a URL
Network policy	when private network targets are blocked, which is the shipped default, the host resolves the host name and refuses private addresses: an SSRF guard that stops a URL from being turned into a probe of the local network
Path policy	fully-qualified local paths only; UNC paths and reparse-point tricks rejected; reserved Windows device names such as CON, PRN, AUX, NUL, the COM ports and the LPT ports rejected, including with extensions and trailing spaces or dots; colon and parent-directory traversal rejected
Roots	allowed roots are normalised and reported; restriction is off by default because this is a single-user tool, and that default is documented in the options class itself
Size cap	the maximum size of a single file defaults to 100 GiB, a backstop rather than a target
Child processes	their own closed STDIN, a bounded time budget, a tree kill on timeout, and an optional lowered priority class that grandchildren inherit
At rest	the request URL is protected with DPAPI in the current-user scope with fixed entropy, and plaintext buffers are zeroed after use
Overwrite	finalization never overwrites an existing destination file
Honesty	a crash is recorded as an unknown outcome; no synthetic success is ever written

Reading this against the live machine. The sizing rules make the current memory load directly relevant: with memory close to ninety percent used, an automatic slot count should resolve low, and 69.1 GB of free disk is smaller than the 100 GiB byte cap, so the cap alone will not protect the volume from one very large download.

1.9 Test, evidence and verification system

Almost half of the Python code of the project is the verification system, and it is built on a deliberate rule: a test must establish its own environment, must never touch the real data of the owner, and must stop with a named reason rather than a stack trace.

1.9.1 Harness modules

Module	Lines	Role
mcp_client	276	standard-library-only MCP STDIO client driving the real executable exactly as a host would; per-run isolation through prefixed environment variables; a host that will not close is stopped by terminating its Windows Job Object, never with a whole-tree taskkill, which has taken the console of a test down with it
tlamatini	155	bridges to the agents of Tlamatini: screenshots only through Shoter, never an in-process screen grab; the window is brought forward and pinned by Windower, foreground ownership is confirmed read-only, and a photo that cannot be confirmed is recorded as unverified; browsers only through Playwrighter in real headed Chrome
fixtures	130	range-capable loopback HTTP server plus deliberately blocking pages, so a transfer can be interrupted at a known moment; builds a two-second H.264 and AAC MP4 and a VP9 and Opus WebM with FFmpeg
console	112	live coloured console output mirrored, colour-stripped, line by line into the run log, with nothing buffered
preflight	94	one named exit code per broken edge
paths	63	confines every command-line path to the evidence or tests folder, following links and junctions
package marker	2	declares the shared harness package

1.9.2 Named exit codes

Exit code	Meaning
0, 1, 3	pass; test failure; every check passed but at least one required photo is missing
10, 11	executable missing; specification invalid
12, 13, 14, 15	FFmpeg missing; ffprobe missing; yt-dlp unavailable; JavaScript runtime missing
16, 17, 18	network unreachable; Tlamatini missing; Chrome missing
19, 20, 21	original source missing; host refused; unsafe command-line path

1.9.3 The suites

Verifier	Lines	What it proves
my videos	415	downloads the own videos of the owner, all submitted at once so the slots are exercised on real data, and then checks every finished file against three independent witnesses
local	597	local integration suite over real MCP STDIO: site extraction, hardware-sized slots, child priority, a real unclean crash in the middle of a transfer, and first-use acquisition of a checksum-verified yt-dlp, while touching neither the real history nor any public video site
tlamatini load	163	loads the specification through the importer of Tlamatini and drives the server with the own client
decoupling	160	source scan for any naming of the original program, for COM interop and for a vendored library; binary scan of the published folder; runtime identity over MCP
comparison	194	four proofs over the comparison document: current, complete, accounted for, and visible
comparison generator	659	generates the comparison deterministically and refuses to run when any file of either project is unaccounted for
photo helper	48	one confirmed full-desktop photo; it exits successfully only when the photo is confirmed

1.9.4 The three witnesses behind every downloaded file

1. **An oracle.** yt-dlp itself, asked with the very same format selector, states which streams should be in the file, including height, codecs and original-language audio, and how long the video is.
2. **The disk.** The test recomputes size and SHA-256, runs an *independent* ffprobe, and performs a full FFmpeg decode of every frame.
3. **The eye.** Playwrighter opens the file in real headed Chrome, plays it, and measures playback through current time, duration, decoded and dropped frames; Windower arranges the console and the browser in front; Shoter photographs the whole desktop. A photo whose window could not be confirmed is not counted.

1.9.5 Recorded outcomes found in the evidence

Suite	Result	What the record shows
build, unit, local	failed, then passed	the failing local run took 143.7 seconds after a 6.0 second build and a 15.0 second unit phase; a later run of the same suite passed in 111.7 seconds from its own suite directory, and both records are preserved side by side, which is what makes the failure inspectable
decoupling	failed in 4.2 seconds	every decoupling check passed except one: the source scan found that the local verifier names the original product inside a docstring on its third line. The check is working; that wording is the offender
my videos	failed, then passed	the passing run verified all nine downloads: repeating a call replays the same job, each job reached a verified completion, the size and the SHA-256 recomputed by the test matched the job, and files were saved under the exact requested names
unit	all passing	path-policy cases including the reserved names CON.txt, CON.foo.bar and NUL, a file name with a trailing space, a trailing dot, an embedded colon, a parent-directory escape, a subdirectory that must be refused, a reserved name with an extension, the empty name, a normal name, the root boundary, UNC rejection and drive-root preservation

One observed defect worth naming. In the earlier my-videos run the audio checks reported that playback was rejected because no user had interacted with the document: Chrome refuses unmuted autoplay without a gesture, and the evidence page relied on autoplay. The later passing run added an explicit click on the play control before measuring playback, and the same audio files then reported measurable playback. That is the evidence system doing exactly what it was built to do: failure and fix are both in the artefacts.

1.10 Provenance: every line beside its counterpart

Stream-Downloader exists to be a mirror, and the project proves that claim mechanically rather than rhetorically.

Mark	Meaning in the comparison document
equal	the line is identical
equivalent	the same line after the systematic rename of the project, its folders and its markers
changed	the line was changed, and the last column says why
removed	a line of the original with no counterpart, with the reason for its removal
added	a Stream-Downloader line with no counterpart, with the reason for its addition

The generated document is 13,351 lines long, and its input fingerprint covers **45 files of the original and 44 Stream-Downloader files**. The generator is deterministic, using no clock and no machine data, and it refuses to produce output when anything is unaccounted for. The verifier regenerates the document in memory and fails on any difference, then parses the Markdown back and asserts that, for every paired file, the line numbers run exactly from one to N on each side and that every decoded line equals the line of that file.

1.11 Design invariants the code enforces

1. **Persist before act.** No side effect precedes the durable record of the job.
2. **Never overwrite.** The verified file is placed without clobbering anything.
3. **Idempotency by normalised request.** Repeats replay; they do not duplicate.
4. **Stage on the destination volume,** so the final move stays same-volume.
5. **One host per state directory.** Exclusive ownership is enforced, not requested.
6. **Children are disciplined:** closed STDIN, bounded time, tree kill, inherited priority.
7. **Concurrency follows the machine, and says why.** Slots are derived and the reason is surfaced.
8. **Inputs are constrained:** scheme, credentials, length, private networks, UNC and device names.
9. **Secrets never rest in the clear.** The request URL is DPAPI-protected at rest.
10. **States never lie.** An unknown outcome is the alternative to a false success.
11. **STDOUT is protocol.** Logging goes to STDERR, always.
12. **Private means private.** No repository, no remote, no upload and no published page.

1.12 Reading the live system metrics against this design

The system context supplied with this document reports CPU 12.9 percent, memory 89.6 percent and 69.1 GB of free disk. Mapped onto the code:

- **CPU is not the constraint here.** At roughly one eighth of capacity, the small-CPU rule is the relevant one, and it is about protecting desktop responsiveness during a merge rather than about starvation.

- **Memory is the constraint.** The low-memory rule exists because simultaneous downloads and merges, with the FFmpeg merge of separate video and audio streams being the expensive case, are what run a machine out of memory. With memory close to ninety percent used, leaving the concurrent-download count automatic is the right call, and the reason string reported by the capabilities tool will show which rule fired.
- **Disk headroom is smaller than the byte cap.** 69.1 GB free is well under the 100 GiB backstop, and because per-job staging lives on the destination volume, a single very large download consumes free space twice before the move completes.

1.13 Known limits, observations and open items

- **Stale absolute paths in recorded artefacts.** The specification file, the build manifest, the test-evidence JSON, the agent runtime configurations and the generated comparison all still point at the original development root, while the tree now lives under an applications folder of Tlamatini. Re-running the suites unchanged against those recorded paths would target a root that no longer holds the build.
- **One decoupling check is red for a documentation reason, not a code reason.** The scan flags the original product name inside a docstring of the local verifier. The source itself is clean; the wording is the offender.
- **Autoplay needs a gesture.** Chrome refuses unmuted autoplay, which is why the evidence pages carry an explicit play control and why the later suite clicks it.
- **Unknown storage is deliberately optimistic.** A volume whose storage class cannot be identified is not treated as a spindle, so it will not be throttled on a guess, and that is a documented trade-off.
- **The comparison document is large by construction.** More than thirteen thousand lines is the price of proving provenance line by line; it is generated and never hand-edited.
- **Scope.** Interactive logins, protected media and multi-user operation are outside the contract, and no support for them should be inferred from the tool names.

1.14 Appendix A: file inventory at a glance

Area	Lines	Contents
Server source	2,734	eleven C# files under the MCP host project folder
Unit tests	results only	the unit test project, wired in through the internals-visible attribute
Python harness	832	seven standard-library modules under the test harness folder
Python verifiers and tooling	2,236	five verifiers, the comparison generator and the photo helper
PowerShell	171	build, suite and protocol-verification scripts
Documentation	see below	a read me of 317 lines, a design of 192, contributor rules of 133, a change log of 56, and the generated comparison of 13,351 lines with its HTML twin
Configuration and build	see below	settings 27 lines, project file 30, common build properties 10, SDK pin 8, locked package graph 344, build manifest 139

1.15 Appendix B: glossary

Term	Meaning here
Job	a persisted download request with an identity, a state and a fingerprint
Staging	a per-job directory on the destination volume where the media lands before verification
Finalization	ffprobe verification, SHA-256, the persisted move intent and the never-overwriting move
Sentinel	a machine-readable progress or output marker printed by yt-dlp and parsed by the transfer service
Slot	a concurrency permit sized from the processors, the memory and the storage of the machine
Fingerprint	the normalised request used as the idempotency identity
External MCP	a server a host such as Tlamatini spawns over STDIO and exposes as one wrapped tool name per tool
Oracle	an independent tool, such as yt-dlp, ffprobe, FFmpeg or Chrome, asked to confirm what the server claims

Stream-Downloader 1.0.0, a private single-user Windows x64 MCP server. Created by **Angela**

López Mendoza. Compiled from the project source, its documentation and its dated test evidence.